ARC-AGI REPL STRATEGY

Evidence Capture & Base-Notebook Integration

How to use successes and failures, what to save, how to integrate it safely, and what to build next



Table of Contents

1. Executive Summary
2. What REPL Means in This Project
3. What We Will Do Inside the REPL
4. Defining Success Correctly
5. Failure Taxonomy and How to Use It
6. Permanent Timeout Policy
7. What We Must Save from Every REPL Run
8. Recommended Canonical Run Schema
9. How Successes Will Be Used
10. How Failures Will Be Used
11. Building Compact, Verifier-Grounded Lessons
12. Integration with the Base Notebook
13. Proposed Base-Solver Flow
14. Hardcoding and Leakage Safeguards
15. Candidate Selection and Anti-Overfit Scoring
16. Using Runtime and Throughput Data
17. Persistence and Artifact Safety
18. Regression Fixtures
19. Ground-Truth Verification Pipeline
20. Next Instrumentation Changes
21. Development Roadmap
22. Immediate Next Work
23. Priority Order
24. Success Metrics
25. REPL Experiment Checklist
26. Final Decision

1. Executive Summary

The REPL is not merely a place to execute Python. In this project it is an observable solver laboratory: the model inspects a task, forms hypotheses, runs probes, writes executable candidates, verifies them against all training pairs, studies errors, and eventually generates test outputs.

- Every REPL action is a research event: what was inspected, what was assumed, what code was produced, and what the verifier reported.
- Successful candidates provide reusable operators, invariants, simplicity signals, and verification patterns.
- Failed candidates provide negative evidence, failure categories, corrective actions, timeout patterns, and ranking signals.
- The base notebook will consume generic features, operator priors, provenance, duplicate detection, and runtime statistics — not stored puzzle answers.
- REPL outputs must be structured, persistent, versioned, and ground-truth aware before they are used as training or ranking evidence.

2. What REPL Means in This Project

The REPL is an agent-driven iterative loop. The model does not simply produce one final answer. It alternates between observation, code execution, candidate submission, deterministic verification, and refinement.

Task	Inspect	Hypothesis	Probe	Candidate	Verify	Refine / Finish
Input	Grid/object analysis	Testable rule	Python/tool calls	transform(grid)	All train pairs	Retry or generate test

Task input

```
task input
  -> inspect grids and pair relations
  -> form a structural hypothesis
  -> run compact Python probes
  -> write transform(grid)
  -> verify on every training pair
  -> analyze mismatches or accept
  -> generate test output
```

3. What We Will Do Inside the REPL

3.1 Task Inspection

- Extract dimensions, colors, background candidates, connected components, bounding boxes, repeated structures, and separators.
- Compare input-output shape, object count, symmetry, movement, tiling, containment, and color-role changes.
- Separate cross-pair invariants from example-specific noise.

3.2 Hypothesis Generation

- Create independent object-centric, geometry-centric, relation-centric, sequence-centric, and direct-synthesis hypotheses.
- Write each hypothesis in testable form: detection rule, operation, and expected invariant.
- Reject explanations that fit only one training pair.

3.3 Python Probes

- Compute components, histograms, symmetry scores, bounding boxes, color frequencies, and object correspondences.
- Keep outputs compact and place essential values first so truncation does not hide them.
- Avoid expensive or unbounded probe code.

3.4 Candidate Transforms

- Produce deterministic transform(grid) functions.
- Do not mutate inputs; handle empty sets, shapes, indexing, and edge cases safely.
- Minimize unexplained fixed coordinates, dimensions, and color literals.

3.5 Verification and Refinement

- Capture compile status, runtime status, shape correctness, exact-pair count, cell accuracy, and mismatch summaries.
- Deduplicate candidates by observed behavior, not only code text.
- Use partial matches and mismatch masks to decide whether to repair a candidate or branch to a new hypothesis.

4. Defining Success Correctly

Level	Condition	Meaning	Use
Tool success	Probe/command	Tool execution is reliable	Latency and reliability

Level	Condition	Meaning	Use
	completed		metrics
Partial candidate	Accuracy above zero, not exact	Part of the hypothesis is correct	Refinement and ranking data
Train exact	All training pairs are exact	Program explains available demonstrations	Candidate pool and regression
Test runtime success	Code runs on test input	Output is valid, correctness unknown	Submission candidate
Ground-truth exact	Evaluation output matches solution	True generalization success	Verified lesson and benchmark

Critical Distinction

train_all_exact = true does not prove the test answer is correct. Until evaluation ground truth is compared, the test output is an unverified prediction.

5. Failure Taxonomy and How to Use It

Failure Type	Signal	Likely Cause	Next Action
HTTP generation timeout	Read timed out	Model response is long or HTTP budget is too small	Separate HTTP timeout; verify token cap; log latency
Python tool timeout	Tool execution timeout	Probe is too heavy or unbounded	Simplify probe; enforce operation limits
No candidate generated	candidate_count = 0	Repeated analyzer timeout, loop stagnation, or protocol failure	Inspect timeout wiring, repetition, and parser
Compile failure	compile_ok = false	Syntax, imports, or function contract error	Save error class and auto-repair prompt
Runtime failure	runtime_ok = false	Index, shape, type, or empty-object bug	Save traceback and failing pair
Shape mismatch	Wrong dimensions	Incorrect transform family or resize rule	Add shape predictor and penalty
Partial accuracy	0 < accuracy < 1	Core idea may be right but alignment/color/edge logic is wrong	Use mismatch mask and parent-child refinement
Duplicate behavior	Same behavior signature	Different code produces the same outputs	Suppress duplicate and spend budget on another branch
Train exact, test	Train passes,	Overfit or ambiguous hypothesis	Use anti-overfit selector and

Failure Type	Signal	Likely Cause	Next Action
wrong	ground truth fails		diversity

6. Permanent Timeout Policy

At least three timeout domains exist and must never share one source variable.

Variable	Purpose	Typical Value	Failure When Wrong
PYTHON_TOOL_TIMEOUT_SECONDS	Local Python/REPL execution	30 s	Heavy probe is interrupted
HTTP_REQUEST_TIMEOUT_SECONDS	vLLM analyzer generation	Measured, e.g. 600 s	Model response is aborted
CANDIDATE_TIMEOUT_SECONDS	Candidate transform verification	30 s	Slow transform is rejected

```
PYTHON_TOOL_TIMEOUT_SECONDS = 30
HTTP_REQUEST_TIMEOUT_SECONDS = 600
CANDIDATE_TIMEOUT_SECONDS = 30

os.environ["LOCAL_ANALYZER_TOOL_TIMEOUT"] = str(
    PYTHON_TOOL_TIMEOUT_SECONDS
)

runner = ARC2Runner(
    ...,
    request_timeout_seconds=HTTP_REQUEST_TIMEOUT_SECONDS,
    candidate_timeout_seconds=CANDIDATE_TIMEOUT_SECONDS,
)
```

Mandatory Guard

The HTTP timeout must never be read from LOCAL_ANALYZER_TOOL_TIMEOUT. Print the effective configuration and assert that HTTP and Python tool timeouts are wired separately before every batch.

7. What We Must Save from Every REPL Run

Artifact	Required Fields	Why It Matters	Current Status
Run record	Task metadata, config,	Reproducibility	Partly available

Artifact	Required Fields	Why It Matters	Current Status
	start/end, status		
Action timeline	Step, command, latency, result, retry	Diagnose stagnation	Must add
Candidate record	Code, parent, step, hypothesis, scores	Provenance and ranking	Incomplete
Train verification	Compile/runtime, pair scores, mismatch	Exact evidence	Available
Behavior signatures	Train and test signatures	Deduplication and ambiguity	Train exists; add test
Test prediction	All test grids, shape, runtime	Submission and evaluation	Available
Ground-truth score	Exactness and cell accuracy	Real quality	Must add
Runtime metrics	Tokens, latency, timeout, retry, GPU	Performance tuning	Partial
Compact lesson	Features, abstraction, failures, operators	Retrieval and generic learning	Must add
Regression fixture	Task, code, expected behavior	Safety after code changes	Must add
Request logs	Prompt/completion metadata	Token and timeout diagnosis	Normalize
Full transcript	All actions and results	Forensic record	Available

8. Recommended Canonical Run Schema

```
{
  "task_id": "...",
  "split": "evaluation",
  "configuration": {
    "model": "...",
    "tool_steps": 8,
    "python_tool_timeout_seconds": 30,
    "http_request_timeout_seconds": 600,
    "candidate_timeout_seconds": 30,
    "tool_output_tokens": 1024,
    "max_output_tokens": 4096
  },
  "run": {
    "status": "completed",
    "accepted": true,
    "analysis_steps": 12,
    "elapsed_seconds": 900.4,
    "timeout_count": 1,
```

```
    "retry_count": 1
  },
  "candidates": [
    {
      "candidate_id": "c003",
      "parent_candidate_id": "c002",
      "analysis_step": 9,
      "hypothesis": "...",
      "code": "def transform(grid): ...",
      "compile_ok": true,
      "runtime_ok": true,
      "all_exact": true,
      "exact_pairs": 3,
      "mean_cell_accuracy": 1,
      "behavior_signature_train": "...",
      "behavior_signature_test": "...",
      "duplicate_behavior": false,
      "failure_category": null
    }
  ],
  "final": {
    "selected_candidate_id": "c003",
    "test_prediction_generated": true,
    "test_ground_truth_verified": false
  }
}
```

9. How Successes Will Be Used

- Identify which structural cues triggered the correct operation family.
- Extract invariants shared by every training pair.
- Identify the operator composition that produced exact behavior.
- Measure time, steps, and tool calls needed to reach the first exact candidate.
- Assess code simplicity, literal dependence, and generality.
- Compare multiple train-exact candidates on test behavior.
- Determine whether the correct candidate was generated but later mis-selected.

10. How Failures Will Be Used

- Failures are negative evidence, not disposable noise.
- Partial scores reveal which sub-operation may already be correct.
- Per-pair mismatches reveal example-specific overfitting.
- Compile and runtime errors form an auto-repair taxonomy.

- Repeated inspect actions or repeated timeouts enable loop-stagnation detection.
- Duplicate candidates quantify wasted compute.
- Train-exact but test-wrong cases reveal ambiguity and anti-overfit features.

11. Building Compact, Verifier-Grounded Lessons

Raw transcripts should not be injected directly into prompt memory. They must be converted into compact lessons whose factual claims are supported by task features, candidate code, and verifier results.

```
{
  "task_features": {
    "input_output_same_shape": true,
    "dominant_separator": "horizontal",
    "component_count_relation": "...",
    "symmetry_candidates": [
      "vertical",
      "horizontal"
    ]
  },
  "successful_abstraction": "...",
  "required_operations": [
    "detect_separator",
    "extract_components",
    "sort_by_size",
    "reflect"
  ],
  "failed_abstractions": [
    {
      "summary": "...",
      "failure_category": "partial_alignment",
      "best_accuracy": 0.81
    }
  ],
  "invariants": [
    "..."
  ],
  "ambiguities": [
    "..."
  ],
  "verified_on_train": true,
  "verified_on_test": false
}
```

Lesson Acceptance Rule

Claims in a lesson must be derived from code, verifier output, and extracted features. Unsupported model narrative remains untrusted metadata.

12. Integration with the Base Notebook

REPL evidence should enter the base notebook through modular, task-independent layers. Original Duck code should remain unchanged unless a modification is truly necessary; instrumentation belongs in the ARC2 adapter/session layer.

Layer	Role	How REPL Data Is Used	Hardcoding Risk
Feature extractor	Describe task structure	Learn which generic features matter	Low
Operator library	Reusable primitives	Mine recurrent verified operations	Low if task-independent
Retriever	Fetch structurally similar lessons	Use compact lessons and feature vectors	Medium; task ID forbidden
Candidate generator	Produce independent hypotheses	Use failure taxonomy and operation priors	Low
Verifier	Deterministic scoring	Use regression fixtures	Low
Selector	Rank train-exact candidates	Use complexity, literal, diversity, GT statistics	Medium; leakage guard required
Budget manager	Allocate steps/tokens/time	Use runtime metrics and timeout history	Low
Artifact exporter	Normalize evidence	Use all run outputs	None

13. Proposed Base-Solver Flow

```
task
  -> extract_structural_features(task)
  -> retrieve_generic_lessons(features, no_task_id=True)
  -> propose_independent_hypotheses(
        object_centric,
        geometry_centric,
        relation_centric,
        sequence_centric,
        direct_synthesis
     )
  -> synthesize candidates
  -> verify every candidate on all training pairs
  -> deduplicate by behavior signature
  -> rank train-exact candidates with anti-overfit score
  -> execute selected candidates on test
```

```
-> save full provenance and metrics
```

14. Hardcoding and Leakage Safeguards

- Task ID must never be used as a solver feature.
- A saved final transform must not be executed directly on a new task.
- Specific coordinates or dimensions may exist inside one candidate, but not as global solver rules.
- Color IDs must not become global rules without a stable semantic role across examples.
- If evaluation ground truth is used for development, a separate held-out validation pool is required.
- No improvement claim should be based only on tasks used to tune the selector or configuration.
- Retrieved lessons should contain abstractions and operator families, not exact answer grids.

15. Candidate Selection and Anti-Overfit Scoring

Signal	Desired Property	Effect
Cross-pair consistency	Same mechanism explains all pairs	Reward
Program simplicity	Few branches and special cases	Reward
Unexplained literals	Fixed row/column/dimension dependence	Penalty
Color-role consistency	Semantic color role is stable	Reward
Object correspondence	Clear input-output mapping	Reward
Test-shape plausibility	Matches learned dimension relation	Reward
Behavior diversity	Different from other exact candidates	Diversity reward
Duplicate signature	Same outputs as previous candidate	Strong penalty
Code size / complexity	Unnecessarily large implementation	Penalty

16. Using Runtime and Throughput Data

- Use per-request prompt tokens, completion tokens, and latency to set real budgets.
- Estimate HTTP timeout from observed latency distribution, not arbitrary constants.
- Use timeout count and repeated action index for stuck-loop detection.
- Use time-to-first-exact-candidate for early stopping.

- Measure duplicate-candidate time as wasted compute.
- Use tool execution duration to cap probe complexity.
- Use GPU KV-cache and queue metrics to distinguish server bottlenecks from long model generations.

```
recommended_http_timeout = max(
    minimum_timeout,
    observed_p95_request_latency * safety_factor,
)
```

17. Persistence and Artifact Safety

Kaggle /kaggle/working is ephemeral. Every batch must be archived and copied to persistent output or a dataset.

```
knowledge/
  runs.jsonl
  candidates.jsonl
  lessons.jsonl
  metrics.jsonl
  test_predictions.jsonl
  solutions.jsonl
  regression/
  raw_runs/
  manifests/
```

- Each archive includes a manifest, timestamp, model ID, notebook version, and configuration hash.
- Interrupted and failed runs are preserved with explicit status.
- Raw and normalized artifacts remain separate.
- The exporter is idempotent: importing the same run twice does not create duplicates.

18. Regression Fixtures

A regression fixture is not a solver shortcut. It verifies that a code change did not break previously working behavior.

```
{
  "task_id": "...",
  "task_payload": {},
  "candidate_code": "def transform(grid): ...",
  "expected_train_outputs": [],
  "expected_behavior_signature_train": "...",
```

```
  "expected_test_prediction": [],
  "verification_version": "..."
}
```

- Task ID may exist as fixture metadata, but never as a runtime solver feature.
- Regression score is not a leaderboard generalization score.
- Fixtures should run after changes to feature extraction, verification, parsing, and candidate execution.

19. Ground-Truth Verification Pipeline

1. Match evaluation challenge and solution files by task ID.
2. Validate test-case count and output shapes.
3. Compute exact equality and cell accuracy.
4. Create a separate class for train-exact but test-wrong cases.
5. Check whether a correct candidate was generated but not selected by comparing candidate test signatures.
6. Write versioned ground-truth results into lesson and selector datasets.

20. Next Instrumentation Changes

Module	Minimum Output
Action event logger	Action start/end, command, latency, success, error, retry
Candidate provenance logger	candidate_id, parent, step, hypothesis, code, score
Request metric parser	Prompt/completion tokens, finish reason, latency
Timeout classifier	HTTP, Python tool, and candidate verifier separately
Test behavior signature	Detect train-identical but test-divergent candidates
Mismatch summarizer	Per-pair shape, color, and region errors
Lesson exporter	Structured compact abstraction
Artifact auditor	Missing, corrupt, or inconsistent fields

21. Development Roadmap

Phase	Deliverable	Exit Condition
Phase 0 — Stabilize	Timeout guards, server reuse, interruption safety, config print	Timeout domains cannot be mixed
Phase 1 — Capture	All run artifacts, request logs, candidate provenance, metrics	Every candidate is recoverable
Phase 2 — Audit/Export	Canonical JSONL exporter, manifest, persistent archive	Runs are reproducible
Phase 3 — Verify	Evaluation ground-truth comparison	Train and test quality are separate
Phase 4 — Diagnose	Success/failure taxonomy; generator vs selector failures	Main causes are quantified
Phase 5 — Features	Generic structural feature extractor	Task-independent feature vector is stable
Phase 6 — Operators	Reusable operator library and compositions	Operators pass regression tests
Phase 7 — Diversity	Independent reasoning branches	Branch contributions are measurable
Phase 8 — Selection	Anti-overfit ranker and behavior diversity	Correct-candidate selection rate improves
Phase 9 — Regression	Broad task-family regression suite	Code changes are regression-safe
Phase 10 — Benchmark	Baseline vs improved ablations	Held-out exact rate and efficiency improve

22. Immediate Next Work

1. Preserve timeout and action-repetition evidence from current runs; do not delete failed runs.
2. Set HTTP timeout from observed latency and verify effective max_output_tokens in the transcript.
3. Add candidate provenance logging — the most urgent missing evidence.
4. Run an artifact auditor after each batch and report missing fields.
5. Archive all raw runs to persistent Kaggle output or dataset storage.
6. Build the evaluation ground-truth comparison tool.
7. Classify runs into success, partial, no-candidate, timeout, runtime failure, and overfit categories.
8. Only then begin the generic feature extractor.

23. Priority Order

Priority	Work Item	Reason	Impact
P0	Timeout separation and guards	A wiring mistake can destroy an entire batch	Reliability
P0	Save candidate code and provenance	Without it, failures cannot teach us	Research evidence
P0	Persistent archive	Session reset causes data loss	Reproducibility
P1	Ground-truth verification	Separates train fit from true correctness	Real quality
P1	Artifact exporter	Turns raw logs into usable data	Automation
P1	Failure taxonomy	Shows where to improve	Targeted fixes
P2	Feature extractor	Foundation for generic reasoning	Generalization
P2	Operator library	Reliable program composition	Solve rate
P2	Selector improvement	Chooses the right exact candidate	Leaderboard quality

24. Success Metrics

- Train-exact task rate
- Evaluation test-exact task rate
- Correct candidate generated rate
- Correct candidate selected rate
- No-candidate rate
- HTTP/Python/candidate timeout rate
- Median and p95 runtime
- Tokens per solved task
- Time to first exact candidate
- Duplicate behavior rate
- Regression pass rate
- Train-exact but test-wrong rate

25. REPL Experiment Checklist

☐ Server health endpoint is ready

- ☐ Model ID matches
- ☐ Python tool timeout is printed
- ☐ HTTP request timeout is printed
- ☐ Candidate timeout is printed
- ☐ Effective max output tokens are verified
- ☐ Effective tool output tokens are verified
- ☐ Task IDs exist
- ☐ Run directory is unique
- ☐ Request logs are enabled
- ☐ Candidate provenance is enabled
- ☐ Progress JSON uses atomic writes
- ☐ KeyboardInterrupt saves progress
- ☐ Batch artifacts are archived
- ☐ Ground-truth status is explicit

26. Final Decision

The main value of the REPL is not the final answer alone; it is the verified trajectory that produced the answer. Every probe, hypothesis, candidate, error, retry, exact match, and test prediction should become structured evidence.

Successes show what works. Failures show whether reasoning, code, verification, timeout control, or candidate selection failed. This evidence can strengthen the base solver without hardcoding any solved puzzle.

> **Next Milestone**
> Complete candidate provenance, artifact auditing, and evaluation ground-truth verification. Then begin the generic feature extractor and reusable operator library.